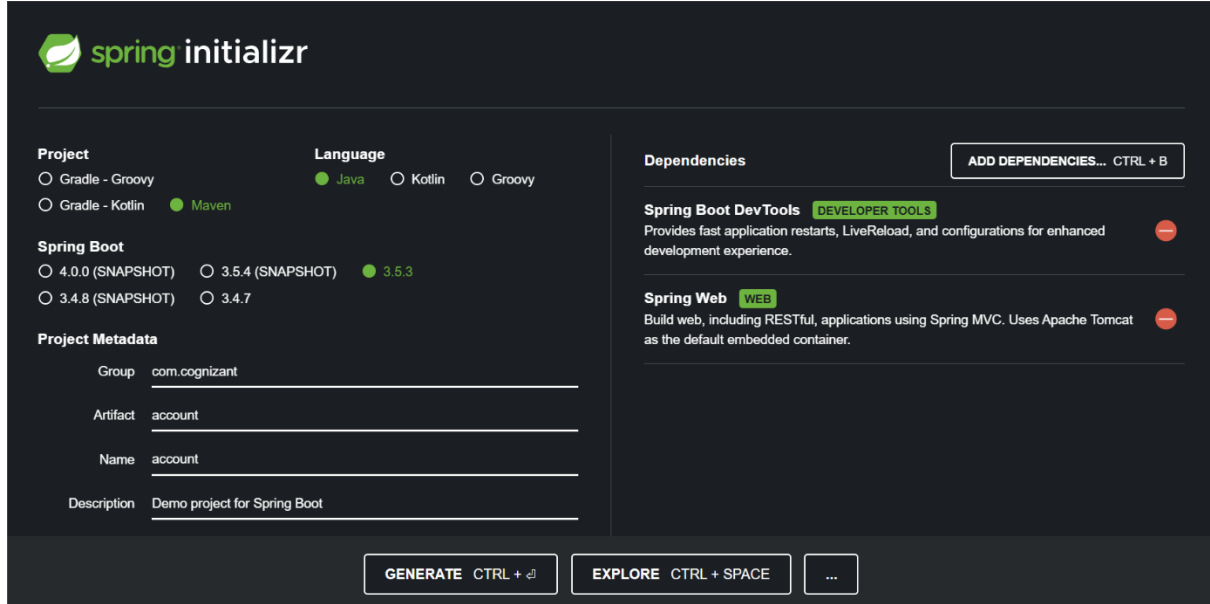


Creating Microservices for account and loan

Account Microservice

1. Project Creation Using Spring Initializer



The image shows the Spring Initializr web interface for creating a new project. The form is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and Action buttons.

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 4.0.0 (SNAPSHOT)
- ☐ 3.5.4 (SNAPSHOT)
- ☒ 3.5.3
- ☐ 3.4.8 (SNAPSHOT)
- ☐ 3.4.7

Project Metadata

Group:

Artifact:

Name:

Description:

Dependencies

[ADD DEPENDENCIES... CTRL + B](#)

Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience. +

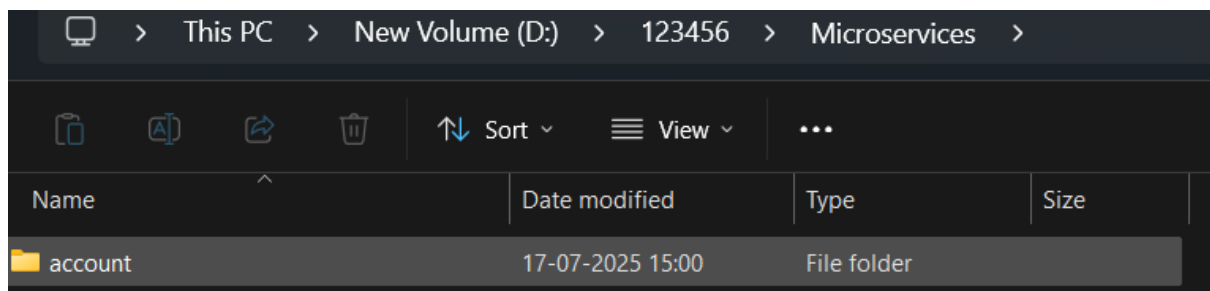
Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container. +

Action Buttons:

- [GENERATE CTRL + G](#)
- [EXPLORE CTRL + SPACE](#)
- [...](#)

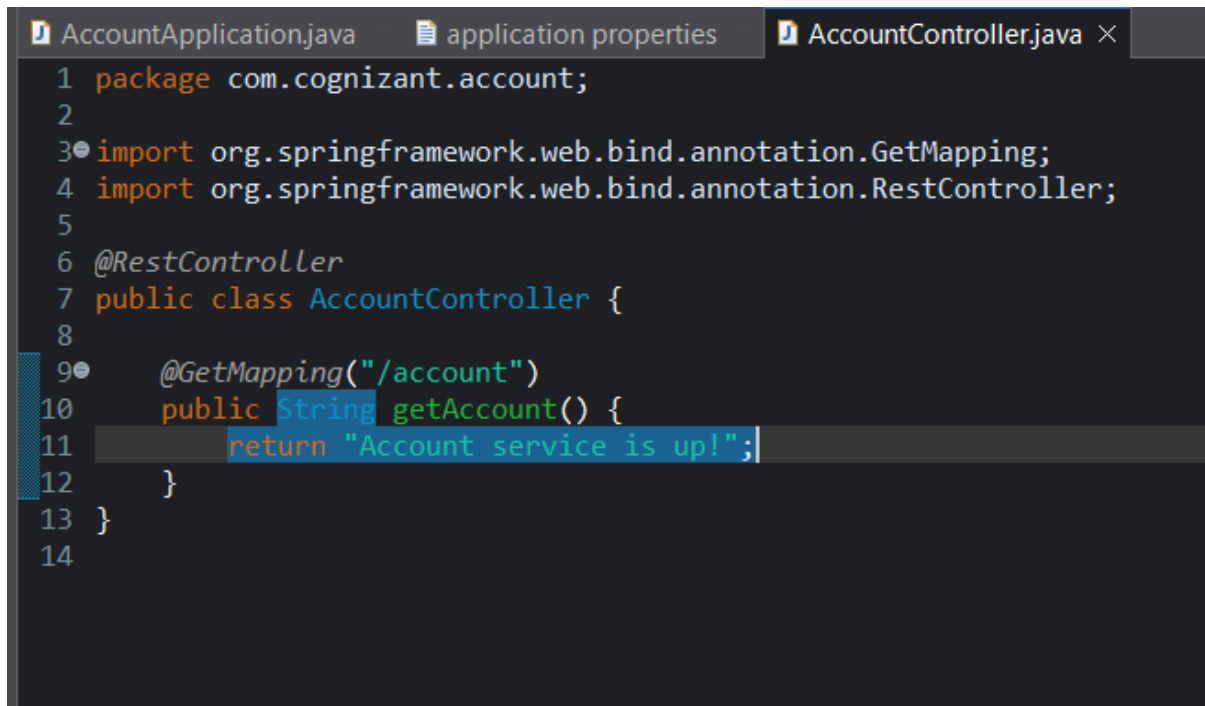
2. Project Setup

- Extract the ZIP.
- Move the account folder into D:\123456\microservices.
- Open command prompt inside D:\123456\microservices\account



3. Import and Code in Eclipse

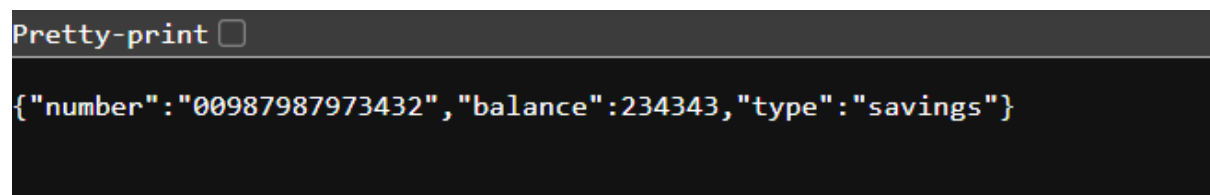
- Import the Maven project into **Eclipse IDE**.
- Inside the com.cognizant.account.controller package, create a controller class



```
AccountApplication.java  application properties  AccountController.java ×
1 package com.cognizant.account;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class AccountController {
8
9     @GetMapping("/account")
10    public String getAccount() {
11        return "Account service is up!";
12    }
13 }
14
```

4. Run and Test

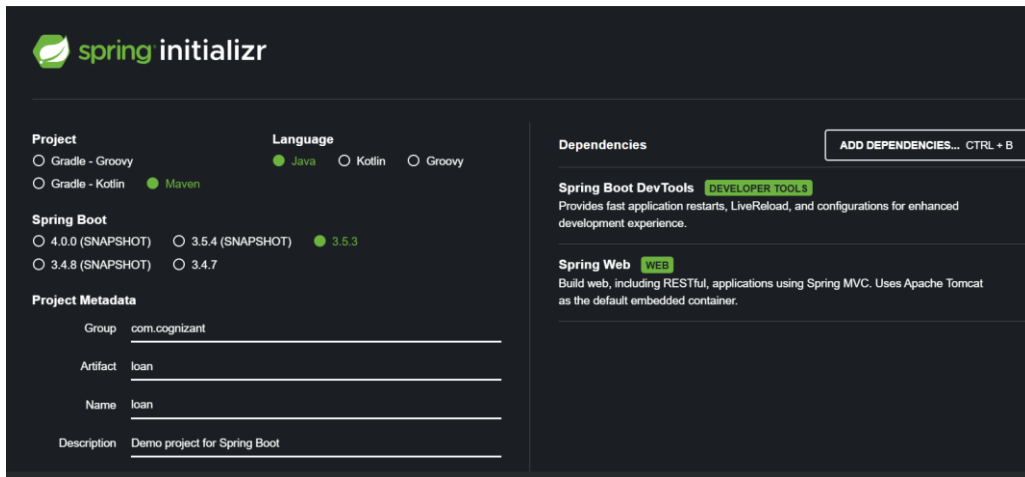
- Run the application.
- Open browser:
<http://localhost:8080/accounts/00987987973432>



```
Pretty-print ☐
{"number":"00987987973432","balance":234343,"type":"savings"}
```

Loan Microservice

1. Create Using Spring Initializr



The Spring Initializr web interface is shown with a dark theme. It includes sections for Project, Language, Spring Boot, Project Metadata, Dependencies, and a list of recommended dependencies.

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.0.0 (SNAPSHOT) ☐ 3.5.4 (SNAPSHOT) ☒ 3.5.3 ☐ 3.4.8 (SNAPSHOT) ☐ 3.4.7

Project Metadata

Group: com.cognizant

Artifact: loan

Name: loan

Description: Demo project for Spring Boot

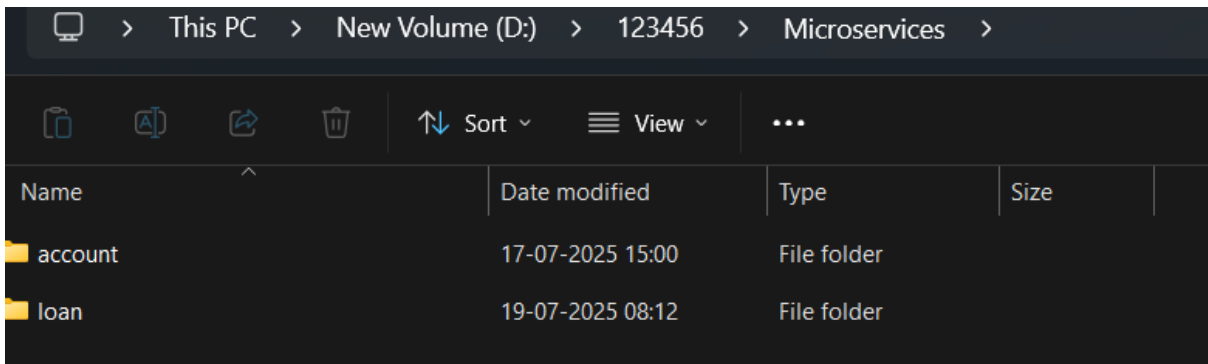
Dependencies

Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

2. Setup and Build

Extract loan folder and move it to D:\123456\microservice

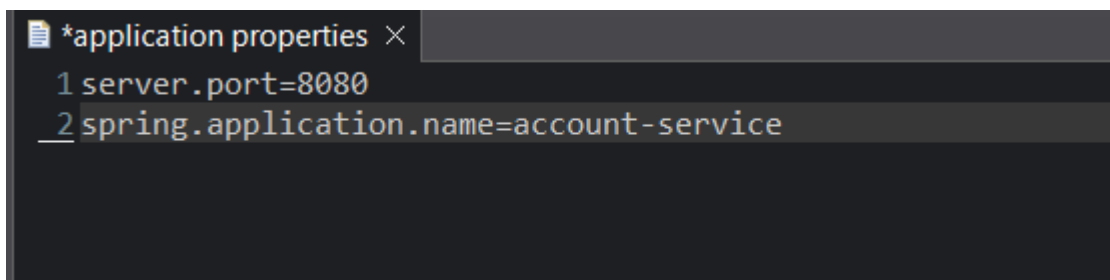


A screenshot of a Windows File Explorer window showing the path D:\123456\Microservices. It contains two folders: 'account' and 'loan'.

Name	Date modified	Type	Size
account	17-07-2025 15:00	File folder	
loan	19-07-2025 08:12	File folder	

3. Update Port

- Edit src/main/resources/application.properties:



```
*application properties ×
1 server.port=8080
2 spring.application.name=account-service
```

4. Implement Controller

- Create a class inside `com.cognizant.loan.controller`:

```
LoanController.java ×
1 package com.cognizant.loan;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4
5
6
7
8
9 @RestController
10 public class LoanController {
11
12     @GetMapping("/loans/{number}")
13     public Map<String, Object> getLoanDetails(@PathVariable String number) {
14         return Map.of(
15             "number", number,
16             "type", "car",
17             "loan", 400000,
18             "emi", 3258,
19             "tenure", 18
20         );
21     }
22 }
23
```

5. Run and Verify

- Keep account service running on port 8080.
- Start loan service (it runs on 8081).
- Open browser:
<http://localhost:8081/loans/12345>

Pretty-print ☐

```
{"loan":400000,"emi":3258,"tenure":18,"number":"12345","type":"car"}
```

